

System Delay

System Delay

1. Learning Objectives
 - SysTick Timer Principle
 - Role of Delay
2. Hardware Setup
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. SysTick Timer Configuration
 3. Write Program
 4. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

1. Learn the basic use of MSPM0G3519 motherboard pins.
2. Understand the usage of the SysTick timer.

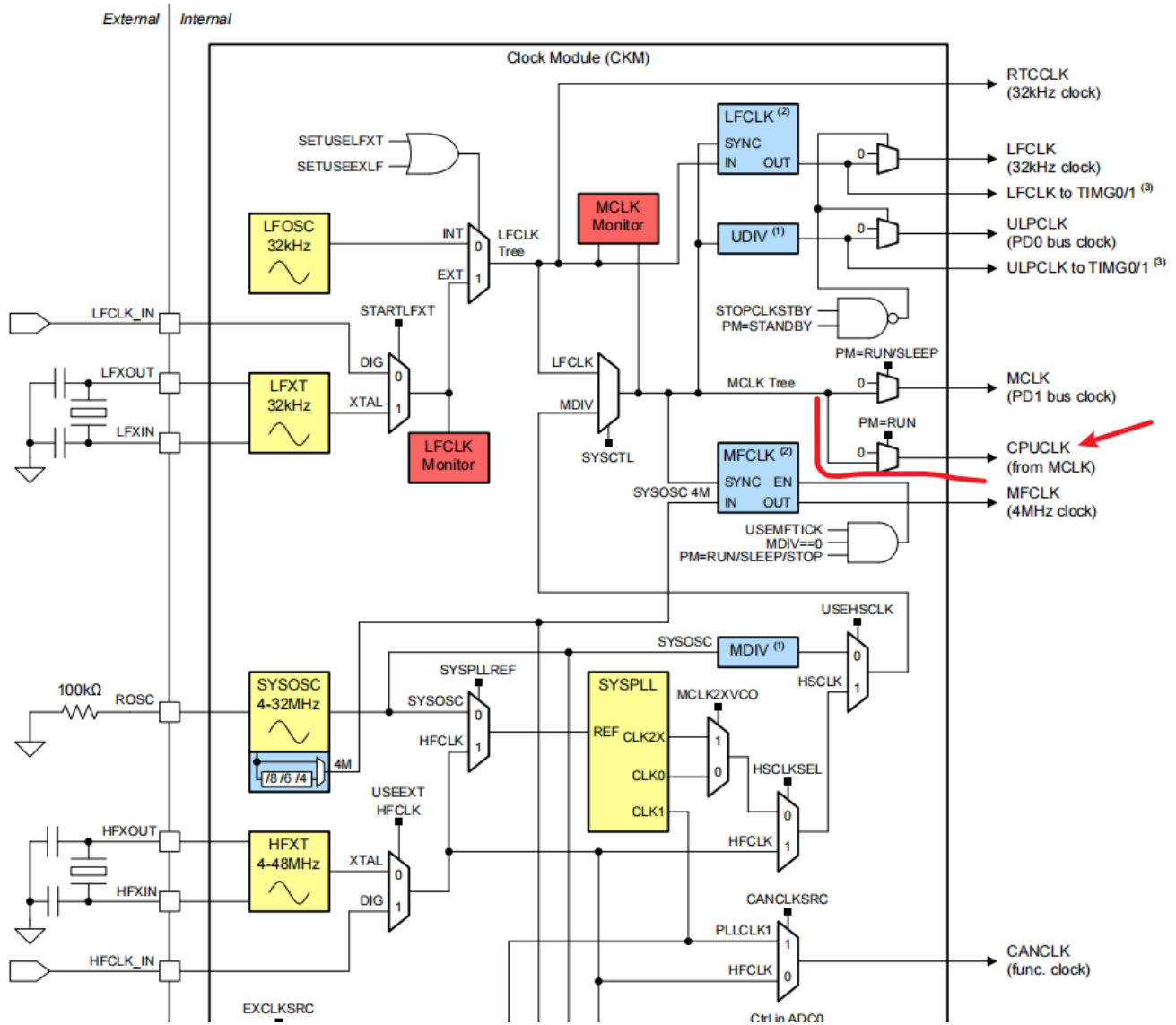
SysTick Timer Principle

The SysTick timer is a commonly used hardware timer used to generate fixed-cycle interrupts or trigger timing operations. Its basic working principle is to count through the main clock (MCLK) to achieve precise time intervals.

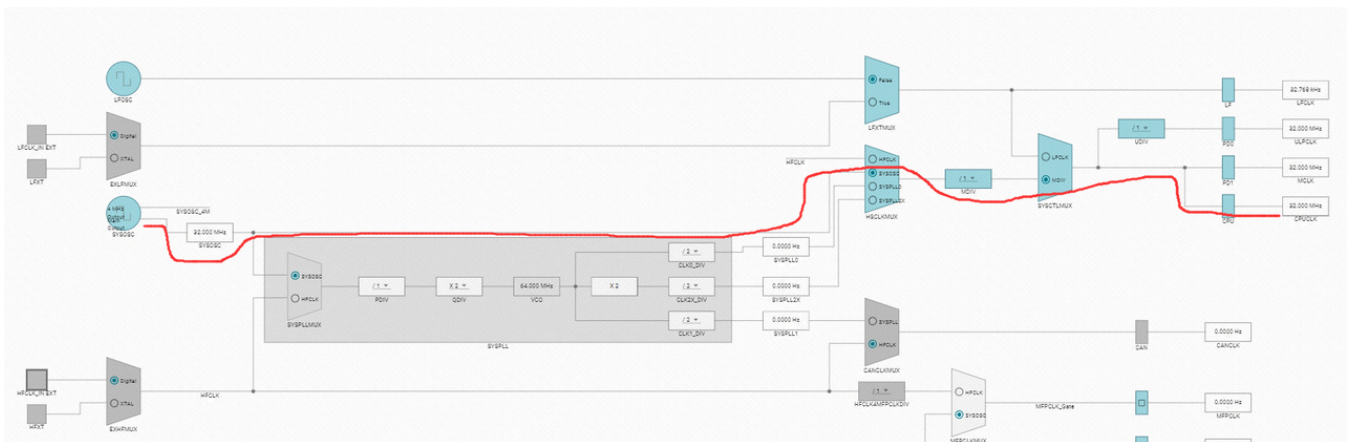
The SysTick timer is a standard timer built into the Cortex-M0 processor. As a 24-bit down counter, it has auto-reload capability and can mask system interrupts. All controllers based on the M0 core are equipped with a SysTick timer, which makes porting programs between different devices simpler. The SysTick timer is typically used in operating systems to provide necessary clock ticks for task scheduling, acting as a "heartbeat" to ensure the system executes tasks on time. Since all M0 core chips contain a SysTick timer, programs do not need to reconfigure the timer when porting, making it easier to port compared to regular timers.

In the system clock configuration, the RCU provides the external clock for the Cortex system timer (SysTick) through MCLK. By default, the MCLK frequency is 32MHz. You can control or read the timer status by setting the SysTick control and status register. For more information about the system clock, refer to the clock tree section in the user manual.

Clock Tree:



Default Clock Path in Project:



The SysTick timer works as follows: After setting the initial value and enabling, each system clock cycle decrements the counter by 1. When the counter reaches 0, SysTick automatically reloads the initial value and continues counting, while setting the internal COUNTFLAG flag to 1 and triggering an interrupt (provided interrupts are enabled).

Role of Delay

A delay function is a function used to introduce an artificial delay in a program. Its main role is to pause the program execution for a specified time period. Simply put, the role of a delay function is to wait - essentially to wait for a period of time before executing the next code.

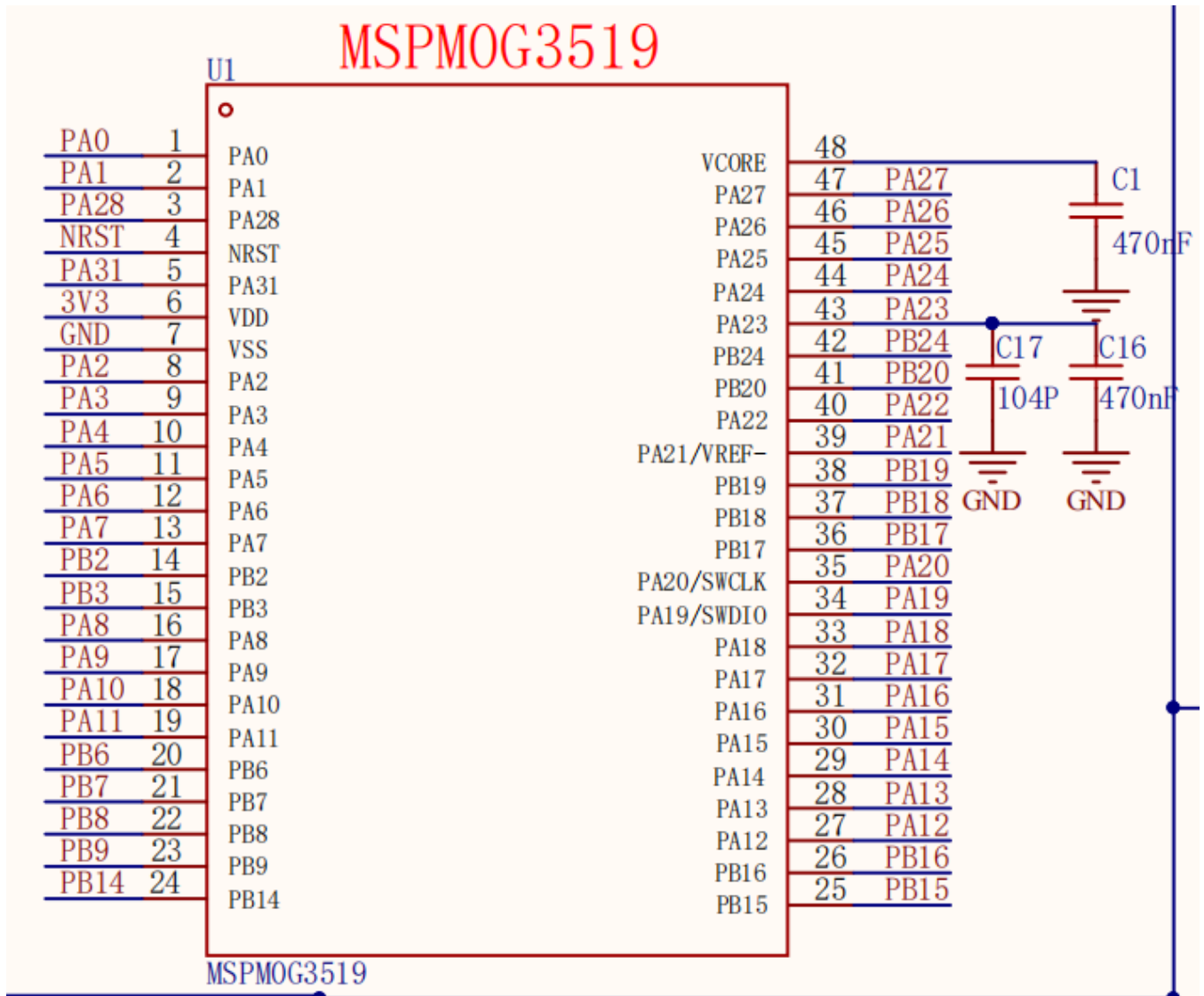
- **Hardware Response Waiting:** Many hardware devices need a certain time to complete operations or stabilize. For example, after sending an initialization command to a display screen, if display data is sent immediately, display abnormalities may occur. Delays allow hardware sufficient time to complete initialization and ensure proper operation.
- **Implementing Timed Tasks:** Delays help implement operations at fixed time intervals. For example, an automatic door controlled by a microcontroller checks infrared sensors for obstacles every 5 seconds. If no obstacle is detected, the door stays closed; if detected, the door opens and stays open for 2 seconds, then automatically closes.
- **User Interaction Optimization:** In user interaction scenarios, delays can achieve better experience. For example, when a button is pressed, a delay can implement "debounce" functionality to avoid multiple triggers caused by mechanical vibration. Additionally, in status indicator light switching, delays control the blinking frequency to make the light flicker smoother and easier to observe.

2. Hardware Setup

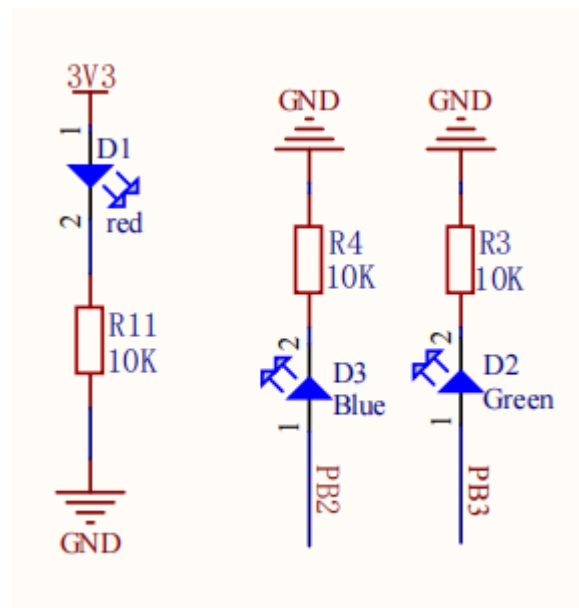
This course does not require additional hardware. You can directly use the onboard LED light on the MSPM0G3519 motherboard. The previous course used **imprecise delay**, and this course uses **SysTick timer to achieve precise delay**.

We have set up two LED user lights on the motherboard. Users can customize their functions. The following uses **LED1** as an example.

MSPM0G3519 Main Control Diagram



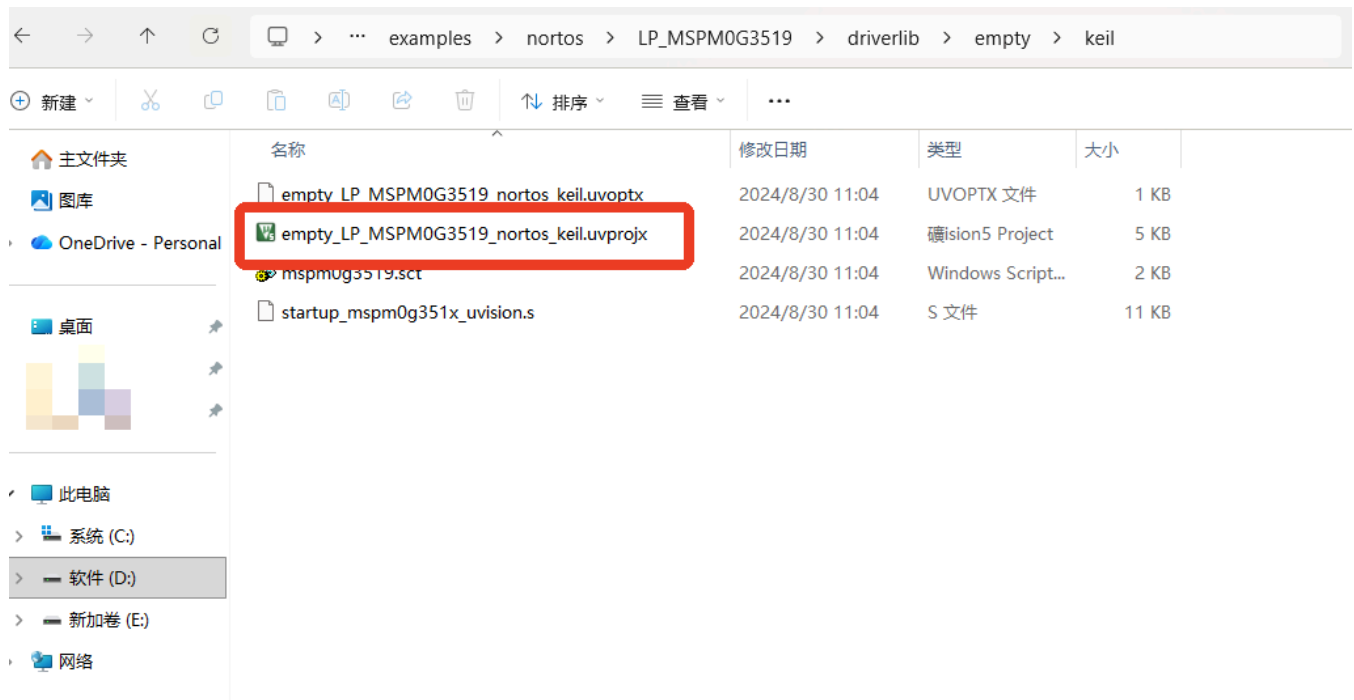
LED Part Schematic



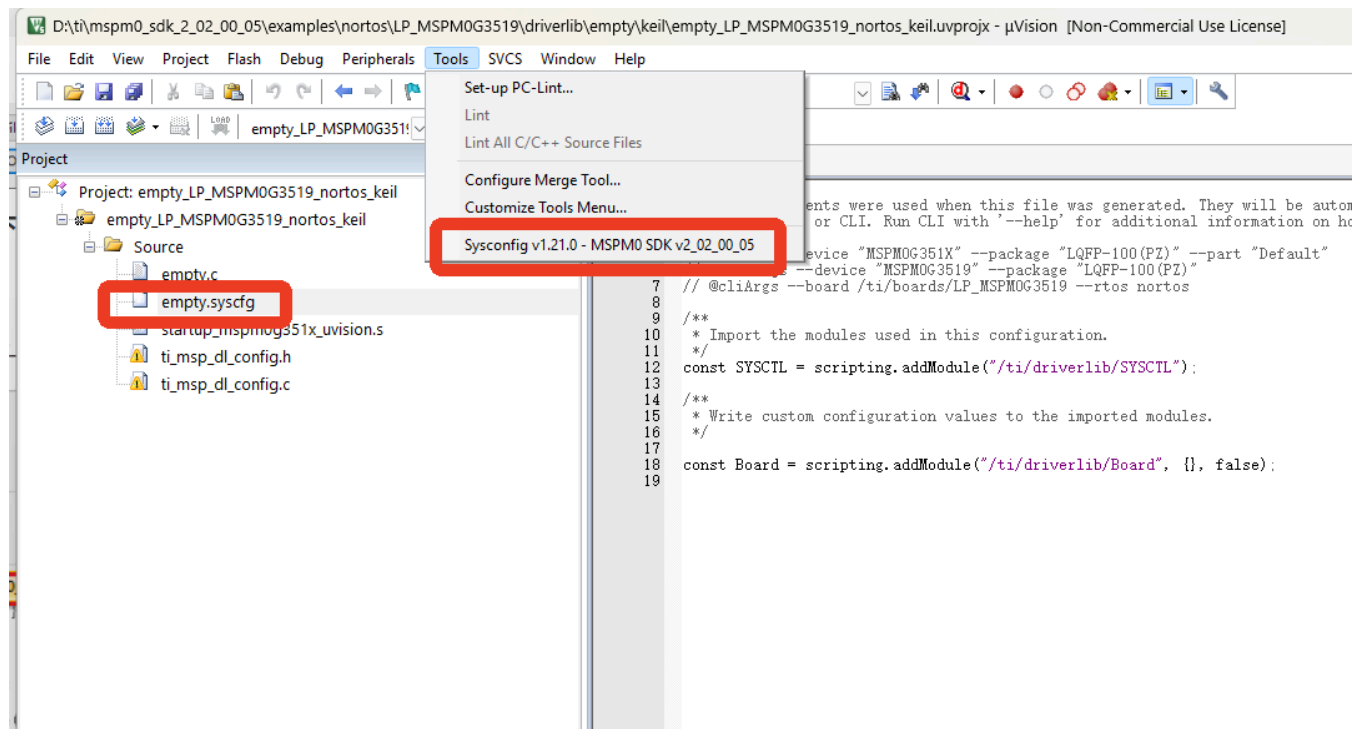
3. Experiment Steps

1. Open SYSCONFIG Configuration Tool

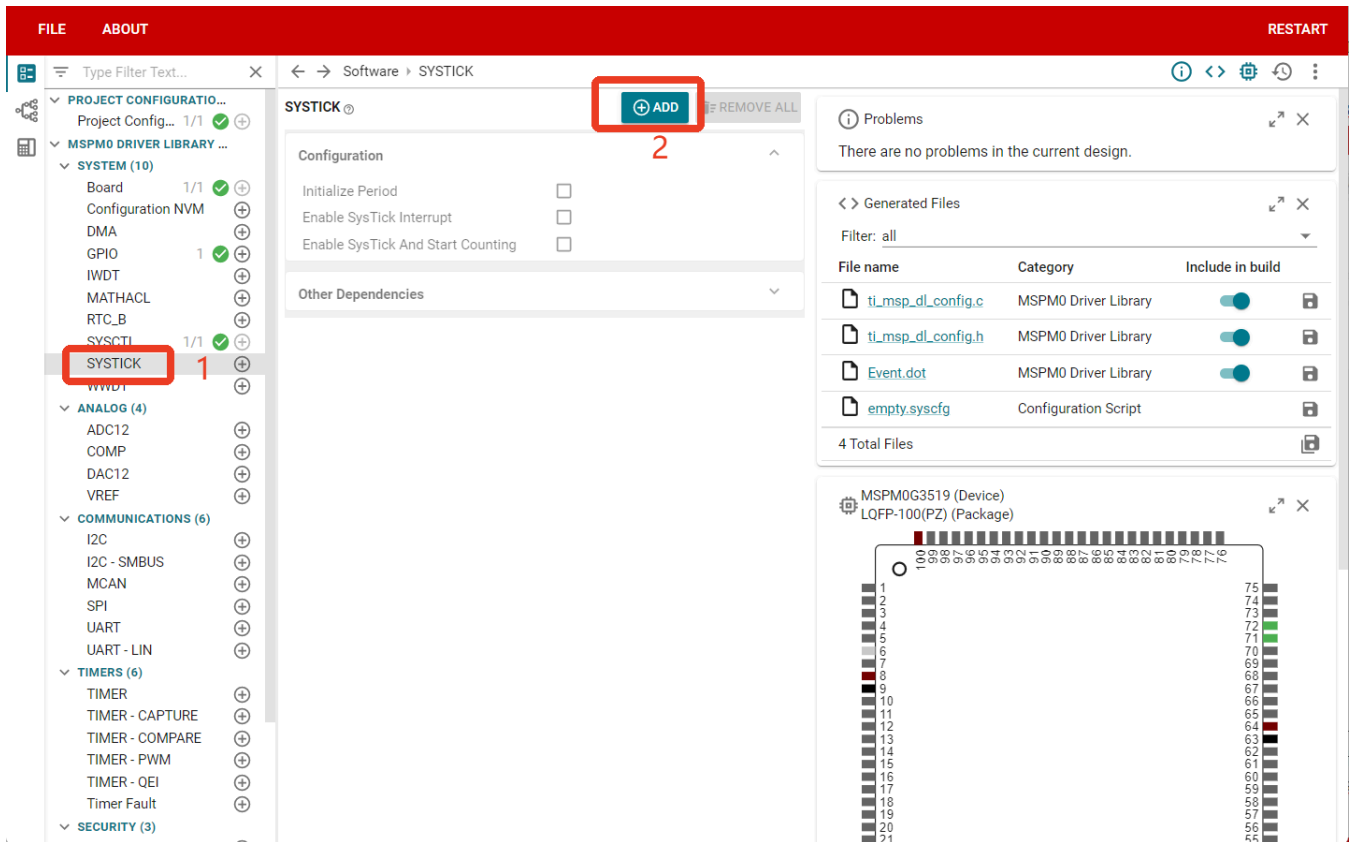
Open the empty project from the SDK in KEIL.



After selecting and opening it, open the empty.syscfg file in the KEIL interface. **With the empty.syscfg file open**, then open the SYSCONFIG GUI interface from the Tools menu.

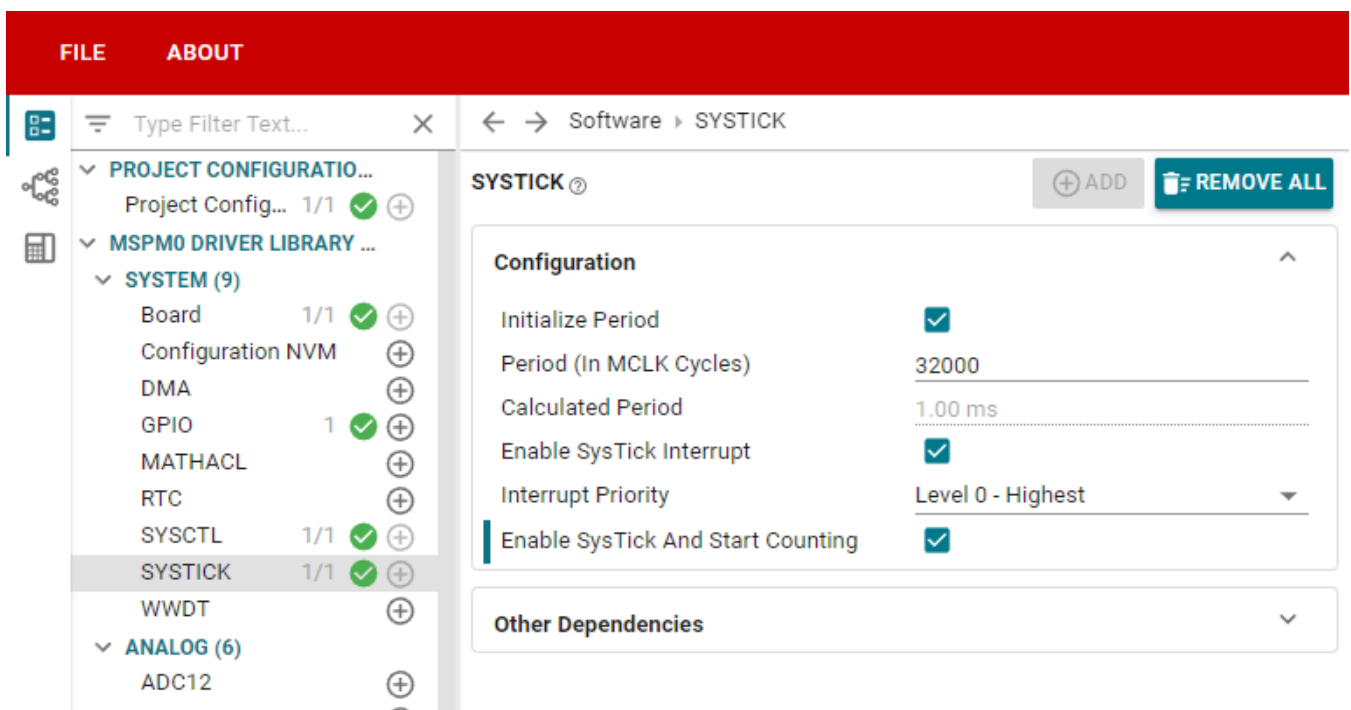


In SYSCONFIG, select MCU peripherals on the left, find the SYSTICK option and click to enter. Click ADD in SYSTICK to add the SysTick timer.



2. SysTick Timer Configuration

The SysTick timer uses MCLK for timing, and MCLK is 32MHz. This means the time for each count is $1/32,000,000 = 0.00000003125$ seconds. Suppose we set the count value to 32000, then the time for counting 32000 values is: $32000 \times 0.00000003125 = 0.001$ seconds = 1ms. Therefore, if we want to set a 1ms SysTick timer, we set the SysTick timer's count value to 32000.



Parameter description:

Initialize Period: Initial timer period.

Period (In MCLK Cycles): Set the periodic count value. Here we set it to 32000.

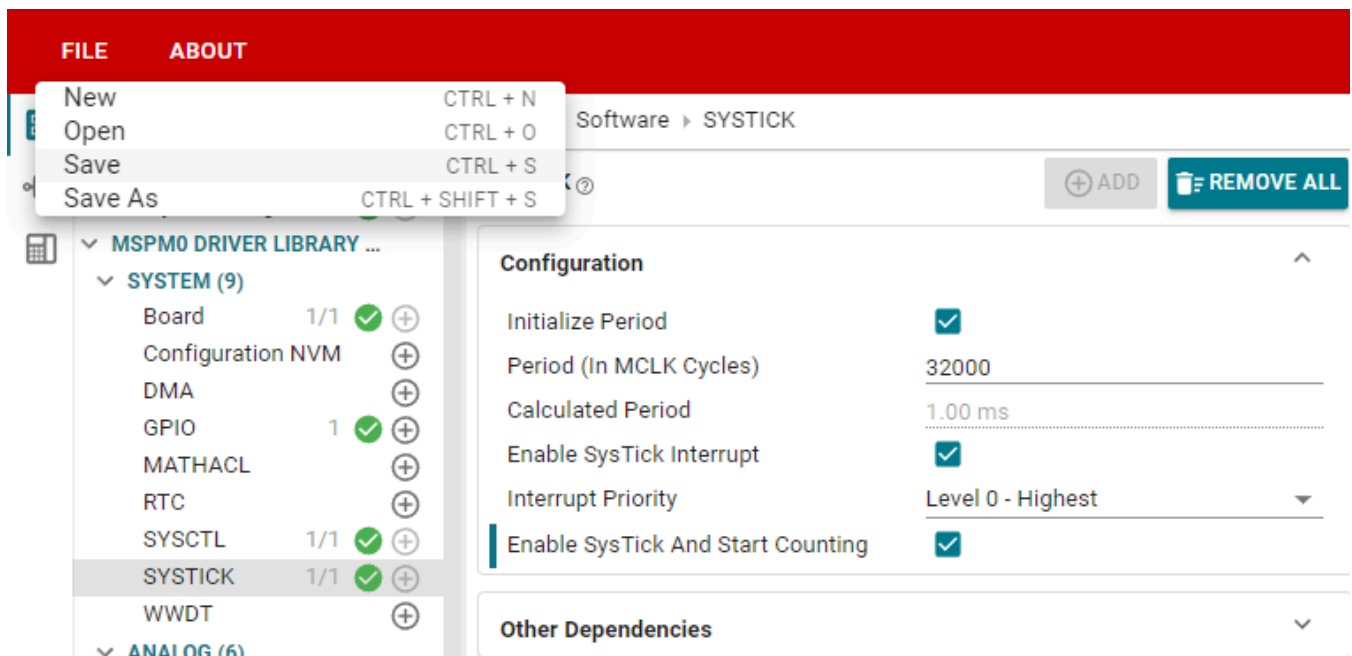
Calculated Period: Time taken for counting. The software automatically calculates this value.

Enable SysTick Interrupt: Whether to enable SysTick timer interrupt.

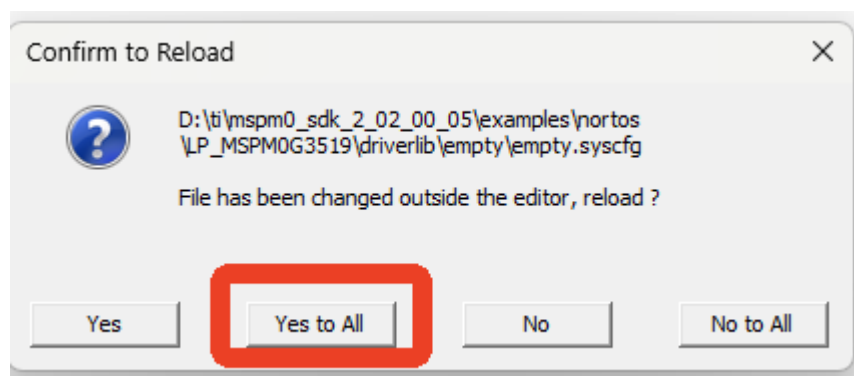
Interrupt Priority: Select interrupt priority. Here the highest priority interrupt is selected.

Enable SysTick And start Counting: Whether to enable SysTick timer and start counting.

Click SAVE to save the SYSCONFIG configuration, then close SYSCONFIG and return to KEIL.



Select **Yes to All** in the popup.



We also need to confirm whether `ti_msp_dl_config.c` and `ti_msp_dl_config.h` files are updated. Simply compile, and the compilation will automatically update to KEIL. If not updated, we can also copy the file content from SYSCONFIG.

3. Write Program

After configuring the SysTick timer, we also need to manually write the SysTick timer interrupt service function. Because we enabled the SysTick timer interrupt, when the SysTick timer's count value decrements from the set initial value to 0, it triggers an interrupt, and after triggering, it automatically executes the interrupt service function. The interrupt service function name for each interrupt is usually fixed and cannot be modified

arbitrarily; otherwise, you will not be able to correctly enter the interrupt service function. The specific name of the interrupt service function can be found in the project's `startup_msp0g350x_uvision.s` file, which lists the interrupt service function names for each interrupt source.

	empty.c	dl_gpio.h	hw_gpio.h	empty.syscfg	startup_msp0g350x_uvision.s
76					DCD HardFault_Handler ; Hard Fault Handler
77					DCD 0 ; Reserved
78					DCD 0 ; Reserved
79					DCD 0 ; Reserved
80					DCD 0 ; Reserved
81					DCD 0 ; Reserved
82					DCD 0 ; Reserved
83					DCD 0 ; Reserved
84					DCD SVC_Handler ; SVCcall Handler
85					DCD 0 ; Reserved
86					DCD 0 ; Reserved
87					DCD PendSV_Handler ; PendSV Handler
88					DCD SysTick_Handler ; SysTick Handler
89					
90					; External Interrupts
91					DCD GROUP0_IRQHandler ; 1: GROUP0 interrupt handler
92					DCD GROUP1_IRQHandler ; 2: GROUP1 interrupt handler

In the empty.c file, write the following code

```
#include "ti_msp_dl_config.h"

volatile unsigned int delay_times = 0;

//Accurate ms delay with tick timer
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

int main(void)
{
    SYSCFG_DL_init();

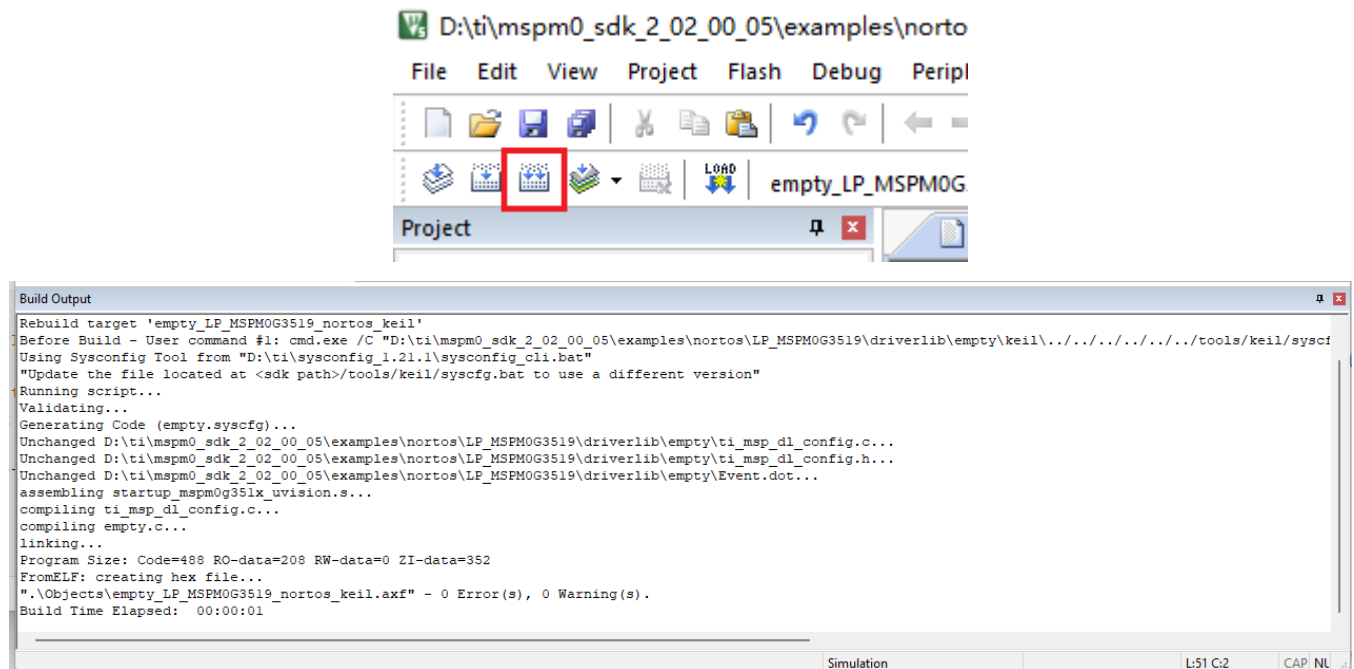
    while (1)
    {
        //LED pin outputs high level
        DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(1000);
        //LED pin outputs low level
        DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(1000);
    }
}

void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}
```

```
}
```

4. Compile

Click the Rebuild icon. The following prompt indicates compilation is complete with no errors.



4. Program Analysis

- empty.c

```
volatile unsigned int delay_times = 0;

//Accurate ms delay with tick timer
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

int main(void)
{
    SYSCFG_DL_init();

    while (1)
    {
        //LED pin outputs high level
        DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(1000);
        //LED pin outputs low level
        DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN);
        delay_ms(1000);
    }
}
```

```
void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}
```

Define a global variable `delay_times` to record the remaining milliseconds for delay.

`delay_ms()` : Implements precise millisecond-level delay.

`main()` : After initializing the system, calls `DL_GPIO_setPins` to specify pins to output high/low level to control LED light on/off.

`SysTick_Handler()` : In the interrupt, decrements `delay_times`. If `delay_times` is non-zero, decrements it by 1 each interrupt. When it decrements to 0, the delay is complete, and the `while` loop in `delay_ms()` exits.

5. Experimental Phenomena

After the program is downloaded, you can see LED1 on the MSPM0 development board light up for 1 second and turn off for 1 second, repeatedly.